A.Define an array and explain its characteristics, advantages, limitations, and applications. Explain the major operations performed on arrays with their time complexities.

1. Definition of an Array

An **array** is a fundamental **linear data structure** that consists of a collection of elements of the **same data type** stored in **contiguous memory locations**. It is classified as a **contiguously-allocated structure** because it is composed of a single slab of memory rather than multiple distinct chunks linked by pointers. Each individual element in an array is identified and accessed by a numerical **index** (or position), which typically starts from 0.

2. Characteristics of Arrays

- **Contiguous Storage:** Elements are placed physically next to each other in memory. If an array starts at address a and each element takes b bytes, the ith element is found at a+b(i−s), where s is the starting index.
- **Homogeneity:** All elements in a particular array must be of the same size and data type.
- **Fixed Size:** Standard arrays have a static size determined at the time of allocation, which cannot be easily adjusted during program execution.
- **Random Access:** Any element can be reached directly in constant time if its index is known.

3. Advantages

- **Constant-Time Access:** Accessing any element takes **O(1)** time.
- **Space Efficiency:** Arrays consist purely of data. Unlike linked structures, no memory is wasted on pointers or links between elements.
- **Cache Locality:** The physical continuity of elements exploits high-speed **cache memory**. Accessing one element often brings nearby elements into the cache, reducing cache misses during sequential processing.

4. Limitations

- **Costly Resizing:** To enlarge a static array, a new, larger array must be created and all existing elements must be copied over.
- **Inefficient Insertions/Deletions:** Adding or removing elements, except at the end, requires shifting multiple other elements to maintain the contiguous order.
- **Wasted Memory:** If the array is not fully utilised, the reserved memory remains occupied and cannot be used by other parts of the program.

5. Major Operations and Time Complexities

Operation	Description	Time Complexity

Access	Directly retrieving an element using its index.	**O(1)**
Update	Changing the value of an element at a specific index.	**O(1)**
Traversal	Visiting each element in the array one by one.	**O(n)**
Insertion	Adding a new element at a specific position; requires shifting subsequent elements to the right.	**O(n)**
Deletion	Removing an element from a specific position; requires shifting subsequent elements to the left to close the gap.	**O(n)**
Linear Search	Scanning the array from the beginning to find a specific value.	**O(n)**
Binary Search	Efficiently finding an item by repeatedly halving the search range; only works on sorted arrays.	**O(logn)**

6. Applications of Arrays

- **Data Storage:** Used for storing and manipulating **tabular data** and records.
- **Mathematical Modeling:** Foundation for **matrix operations**, 2D/3D grids, and numerical simulations.
- **Implementation of Other Structures:** Arrays serve as the underlying building block for **Stacks, Queues, Heaps, and Hash Tables**.
- **Image Processing:** Images are often represented as **2D arrays of pixel values**.
- **Polynomial Representation:** Arrays can efficiently store the coefficients of polynomials for arithmetic operations.

B.Define a stack and explain the LIFO principle. Describe the PUSH, POP, and PEEK operations with suitable algorithms and analyze their time and space complexities

Definition of a Stack

A **stack** is an ordered collection of items where all insertions and deletions are restricted to a single end, commonly referred to as the **top**. It is a linear data structure that serves as a container for elements of a specific type, where access is limited to the most recently added item. Stacks are often described as **"pushdown lists"** because of how elements are added to the structure.

The LIFO Principle

The fundamental principle governing a stack is **Last-In, First-Out (LIFO)**. This policy dictates that the element most recently inserted into the stack is the first one to be removed. Intuitively, this is analogous to a **pile of plates** or books, where one can only conveniently add or remove an item from the top of the pile.

Stack Operations and Algorithms

Stack implementations typically use an **array** or a **linked list**. In an array-based implementation, an integer attribute (often called `top`) tracks the index of the most recently inserted element.

1. PUSH Operation

The **PUSH** operation adds a new element to the top of the stack. Before insertion, the algorithm must verify if there is sufficient space to avoid a **stack overflow**.

Algorithm (Array-based):

```
PUSH(S, x)
1. If S.top == S.size
2.    error "overflow"
3. Else
4.    S.top = S.top + 1
5.    S[S.top] = x
```

Note: In some implementations, `top` is initialized to -1, and the increment occurs before assignment.

2. POP Operation

The **POP** operation removes and returns the top element of the stack. The algorithm must first check if the stack is empty to prevent **stack underflow**.

Algorithm (Array-based):

```
POP(S)
1. If STACK-EMPTY(S)
2.    error "underflow"
3. Else
4.    S.top = S.top - 1
5.    return S[S.top + 1]
```

3. PEEK (or TOP) Operation

The **PEEK** operation returns the value of the current top element **without removing it** from the stack.

Algorithm:

```
PEEK(S)
1. If STACK-EMPTY(S)
2.    error "underflow"
3. Else
4.    return S[S.top]
```

Complexity Analysis

Time Complexity

In both array and linked-list implementations, the primary stack operations are highly efficient because they do not require searching or shifting elements.

- **PUSH:** $O(1)$.
- **POP:** $O(1)$.
- **PEEK:** $O(1)$.
- **IsEmpty:** $O(1)$.

*Note: In dynamic array implementations, the **worst-case** PUSH might take $O(n)$ during resizing, but the **amortized cost** remains $O(1)$.*

Space Complexity

The space complexity is $O(n)$, where n is the number of elements stored in the stack.

- **Instruction Space:** Memory for the compiled code.
- **Data Space:** Memory allocated for the array or the nodes in a linked list.
- **Environmental Stack:** Stacks are also used internally by compilers to manage recursive function calls, where each call requires $O(1)$ stack space, leading to $O(n)$ total space for a recursion depth of n

C.Define a queue and explain the FIFO principle. Describe ENQUEUE and DEQUEUE operations using an array with suitable algorithms and analyze their time complexities.

Definition of a Queue

A **queue** is an ordered collection of items where all insertions are made at one end, known as the **rear (or tail)**, and all deletions are made at the other end, known as the **front (or head)**. It is a linear data structure that serves as a container where the order of retrieval is determined by the sequence in which items arrived.

The FIFO Principle

The fundamental governing rule of a queue is the **First-In, First-Out (FIFO)** principle. This principle dictates that the first element inserted into the queue will be the first one to be removed. Intuitively, this is analogous to a **waiting line** of people at a bank or a ticket counter: the person who arrives first is the first to be served, and new arrivals join the back of the line.

Array-Based Implementation

When implementing a queue using an array, two integer attributes are typically maintained:

- **Front (Head):** Indexes the location of the element to be removed.
- **Rear (Tail):** Indexes the next available location where a new element will be inserted.

To prevent memory wastage, a **circular array** implementation is often used, where the first element of the array is imagined to immediately follow the last element, allowing the head and tail to "wrap around" when they reach the array's end.

1. ENQUEUE Operation

The **ENQUEUE** operation adds a new item to the rear of the queue.

Algorithm (Circular Array):

```
ENQUEUE(Q, x)
1. Q[Q.tail] = x
2. if Q.tail == Q.size
3.    Q.tail = 1
4. else Q.tail = Q.tail + 1
```

[Note: Standard error checking for "overflow" should be performed if `Q.head == Q.tail + 1` or if the array is completely occupied.]

2. DEQUEUE Operation

The **DEQUEUE** operation removes and returns the element at the front of the queue.

Algorithm (Circular Array):

```
DEQUEUE(Q)
1. x = Q[Q.head]
2. if Q.head == Q.size
3.    Q.head = 1
4. else Q.head = Q.head + 1
5. return x
```

[Note: Standard error checking for "underflow" should be performed if the queue is empty (`Q.head == Q.tail`).]

Complexity Analysis

Time Complexity

The primary operations of a queue are highly efficient:

- **ENQUEUE: \(O(1)\)**.
- **DEQUEUE: \(O(1)\)**.
- **Peek/Front: \(O(1)\)**.
- **IsEmpty: \(O(1)\)**.

These operations are constant time because they involve simple indexing and incrementing of pointers without the need to search or shift multiple elements.

Space Complexity

The space complexity is **\(O(n)\)**, where \(n\) is the number of elements the queue is designed to hold. In a static array implementation, this space is pre-allocated regardless of the current number of elements in the queue.

Applications

- **Shared Resource Management:** Managing tasks waiting for a printer, disk storage, or CPU time in a time-sharing system.
- **Breadth-First Search (BFS):** Used as an auxiliary data structure to traverse graphs level by level.
- **Data Streaming:** Buffering data chunks to ensure they are processed in the correct arrival order.

D. Define a circular queue. Explain its representation, working, insertion and deletion operations with suitable algorithms and analyze their time complexities.

1. Introduction

A **circular queue** is an advanced variation of the linear queue designed to optimize memory utilization in array-based implementations. While a standard linear queue suffers from "creeping" indices that eventually render memory unusable, the circular queue creates a continuous loop of storage.

2. Definition

A **circular queue** is a linear data structure that follows the **First-In, First-Out (FIFO)** principle, where the last position of the array is logically connected back to the first position to form a circle.

3. Why/Need

The primary need for this structure is to overcome the **limitations of linear implementations**, where both `rear` and `front` indices only increase. In a linear queue, as items are dequeued, the storage space at the beginning of the array is discarded and cannot be reused, leading to a "false overflow" where the queue is reported as full even if vacant slots exist at the front.

4. Core Concept

The core principle is **wrap-around indexing**. Instead of stopping at the end of the array (index `MAX-1`), the indices reset to `0` using the **modulo operator**: `index = (index + 1) % SIZE`.

5. Representation

It is typically represented using a **static array** of a fixed size along with two integer variables:

- **Front (Head):** Indexes the location of the element to be removed.
- **Rear (Tail):** Indexes the location where the next new element will be inserted.

6. Working / Principle

The circular queue operates like a ring of elements. When an element is enqueued, the `rear` index moves clockwise to the next slot; when an element is dequeued, the `front` index also moves clockwise, effectively freeing the previous slot for future reuse.

7. Algorithms

A. ENQUEUE (Insertion)

1. **Check for Overflow:** If `((FRONT == 0 && REAR == MAX-1) OR (FRONT == REAR + 1))`, display "Queue Full" and exit.
2. **Initial Insertion:** If `FRONT == -1`, set `FRONT = 0` and `REAR = 0`.
3. **Circular Increment:** Else, set `REAR = (REAR + 1) % SIZE`.
4. **Insert Data:** `Q[REAR] = DATA`.

B. DEQUEUE (Deletion)

1. **Check for Underflow:** If `FRONT == -1`, display "Queue Empty" and exit.
2. **Retrieve Data:** `DATA = Q[FRONT]`.
3. **Single Element Case:** If `FRONT == REAR`, reset `FRONT = -1` and `REAR = -1`.
4. **Circular Increment:** Else, set `FRONT = (FRONT + 1) % SIZE`.
5. **Return Data**.

8. Example

Consider an array of size 5 (`0` to `4`). If `f=2` and `r=4` with elements at indices 2, 3, and 4, the queue is not full in a circular model. Enqueuing a new element will wrap the `rear` to index `0`, effectively utilizing the vacant space at the beginning of the array.

9. Advantages

- **Memory Efficiency:** It prevents the wastage of memory locations vacated by the `Dequeue` operation.
- **Predictable Performance:** It provides consistent performance without the need for element shifting.

10. Disadvantages

- **Fixed Size:** Static implementations cannot grow beyond the pre-allocated array size.
- **Complexity:** The logic for overflow and underflow is slightly more complex than a simple linear queue.

11. Complexity Analysis

- **Time Complexity:** Both Enqueue and Dequeue take **O(1)** time as they involve only pointer updates and direct array access.
- **Space Complexity: O(n)**, where n is the maximum number of elements the queue can hold.

12. Important Exam Points

- **Overflow Condition:** `(rear + 1) % SIZE == front`.
- **Underflow Condition:** `front == -1`.
- **Modulo Usage:** Understand that `% SIZE` is the mathematical tool that provides circularity.

- **Capacity:** A circular queue of size n typically stores only $n-1$ elements if one position is left vacant to distinguish between "full" and "empty" states

Differentiate between a linear queue and a circular queue. Why is the latter preferred?

1. Structural and Functional Differences

Feature	Linear Queue	Circular Queue
Logic	Elements form a straight logical sequence.	The last position is logically connected back to the first to form a ring.
Index Movement	Both `front` and `rear` indices increase monotonically and never decrease.	Indices "wrap around" to the beginning of the array once they reach the end.
Overflow Condition	Occurs when the `rear` index reaches the maximum size of the array (`REAR == MAX-1`), regardless of vacant space at the front.	Occurs only when the array is fully occupied, calculated as `(REAR + 1) % SIZE == FRONT`.
Memory Reuse	Space vacated by dequeued elements at the beginning of the array is discarded and never reused.	Vacated space at the front is reused by new elements through the wrap-around mechanism.

2. Operation Mechanics

- **Linear Implementation:** To enqueue an item, the `rear` is increased by one, and the item is placed in that position. To dequeue, the item is taken from the `front`, and the `front` is increased by one. This "creeping" of indices means that after a series of operations, the indices eventually reach the end of the allocated memory slab.
- **Circular Implementation:** This structure uses the **modulo operator** to manage indices. For example, the `rear` is updated using `REAR = (REAR + 1) % SIZE`. This ensures that if incrementing an index causes it to go past the array length, it resets to zero, allowing the process to continue in a circle.

3. Why the Circular Queue is Preferred

The circular queue is preferred in nearly all array-based implementations due to its **superior memory efficiency**.

1. **Elimination of "False Overflow":** In a linear queue, once the `rear` reaches the last index, the queue is considered "full," even if many elements have been dequeued

and the beginning of the array is vacant. The circular queue solves this by allowing the `rear` to wrap back to index 0 to fill those vacant slots.

2. **Predictable Performance:** Linear queues sometimes attempt to solve the overflow problem by shifting all remaining elements to the front every time a deletion occurs. However, this results in an **O(n)** time complexity for deletions. Circular queues maintain **O(1)** for both insertion and deletion without requiring any element shifting.
3. **Resource Management:** Circular queues are ideal for buffering and shared resource management (like CPU scheduling or print buffers) because they provide a continuous loop for incoming tasks without needing to reallocate or reset the data structure frequently.

E.Explain the conditions for **overflow and underflow** in a circular queue. Illustrate enqueue and dequeue operations using a suitable example and diagram

A **circular queue** is a linear data structure that resolves the "false overflow" problem of linear queues by logically connecting the last position of the array back to the first, forming a ring. This allows for efficient memory reuse as vacated spaces at the front are filled by new elements through a wrap-around mechanism.

1. Overflow and Underflow Conditions

In a circular queue implementation of size n, the following conditions define its boundary states:

- **Overflow (Queue Full):** This occurs when the array is completely occupied and an attempt is made to insert a new element. Mathematically, overflow is reached when the next position of the `rear` index (after wrap-around) would equal the `front` index.
    - **Condition:** `((FRONT == 0 && REAR == MAX-1) OR FRONT == REAR + 1)`.
    - Using the modulo operator for circularity, this is often expressed as: `(REAR + 1) % SIZE == FRONT`.
- **Underflow (Queue Empty):** This occurs when there are no elements in the queue and an attempt is made to remove an item.
    - **Condition:** `FRONT == -1 && REAR == -1` (for implementations where indices are initialized to -1).
    - In other implementations, the condition may be `FRONT == REAR`.

---

2. Illustration of Operations

Consider a circular queue with a static array of **size 5** (indices 0 to 4).

A. Enqueue Operation (Insertion)

The `rear` index is incremented circularly to add a new item.

1. **Initial State:** `f = -1, r = -1` (Empty).
2. **Enqueue(32), Enqueue(18), Enqueue(58):**
    - `FRONT` becomes 0, `REAR` moves to 2.
    - The queue contains: `[32, 18, 58, _, _]`.

3. **Wrap-around Case:** If we continue enqueuing until the `rear` reaches index 4 and then perform a circular increment:
    - `REAR = (4 + 1) % 5 = 0`.
    - If index 0 is vacant (due to a prior dequeue), the new element is placed there.

B. Dequeue Operation (Deletion)

The `front` index is incremented circularly to remove and return the first element.

1. **Current State:** `f = 0, r = 2` (Elements: 32, 18, 58).
2. **Dequeue():**
    - Returns 32.
    - `FRONT` increments to 1.
3. **Single Element Case:** If `FRONT == REAR` and a dequeue occurs, both indices are reset to -1 to indicate the queue is now empty.

---

3. Diagrammatic Representation

Based on source materials, a circular queue is visualized as a ring where indices wrap around.

```
      <-- FRONT (Initial position)
   /    \
 [9]    [10]
  |      |
 [11] --- [12] <-- REAR (Moves clockwise)
```

- **Visual Step-by-Step ():**
    - **Full Wrap:** If `f=2` and `r=4` with elements at 2, 3, and 4, the queue is not full. `Enqueue(21)` will move `r` to 0, placing the new value in the vacant slot at the beginning of the array.
    - **Queue Full State:** If `f=0` and `r=4`, the next enqueue triggers the **Overflow** error because `(4+1) % 5 == 0`, which matches the `front`.

Summary of Complexity

- **Time Complexity:** Both Enqueue and Dequeue take $O(1)$ as they involve only direct array access and pointer updates.
- **Space Complexity:** $O(n)$, where n is the fixed size of the allocated array

F.Explain the different types of linked lists and explain its advantages over arrays . Compare singly linked lists, doubly linked lists, and circular linked lists based on structure, operations, advantages, disadvantages, and applications

A **linked list** is a linear collection of data elements called nodes, where the linear order is not determined by their physical placement in memory but by **pointers** that bind distinct chunks of memory together. Each node typically consists of at least two parts: an **information field** (the data) and a **link field** (pointer) containing the address of the next item in the sequence.

Advantages of Linked Lists Over Arrays

Linked lists offer several critical advantages over contiguously-allocated structures like arrays:

- **Dynamic Size:** Unlike static arrays, which have a fixed size determined at allocation, linked lists can grow and shrink dynamically during program execution.
- **Efficient Insertions and Deletions:** In an array, inserting or deleting an element at the beginning or middle requires shifting all subsequent elements, resulting in $O(n)$ time complexity. In a linked list, these operations only require adjusting pointers, which takes $O(1)$ time once the position is reached.
- **No Memory Wastage for Pre-allocation:** Arrays reserve memory for their entire declared size even if only a fraction is used; linked lists only allocate memory for a node when the need arises.
- **Ease of Reorganization:** For large records, moving pointers is significantly faster and computationally cheaper than moving the actual data items in memory.

Comparison of Linked List Types

1. Singly Linked List (SLL)

- **Structure:** Each node contains data and a single **"next"** pointer to the succeeding node; the final node points to **NULL** to signal the end of the list.
- **Operations:** Supports only **forward traversal**. Insertion/deletion at the head is $O(1)$, but deletion at the tail is $O(n)$ because the predecessor of the tail must be found via traversal.
- **Advantages:** Lowest memory overhead per node among all types (only one pointer).
- **Disadvantages:** No backward traversal and inefficient tail operations.
- **Applications:** Implementation of stacks and simple queues.

2. Doubly Linked List (DLL)

- **Structure:** Each node contains data and **two pointers**: **"next"** (to the successor) and **"prev"** (to the predecessor).
- **Operations:** Supports **bi-directional traversal** (forward and backward). Insertion and deletion at both ends are $O(1)$.
- **Advantages:** Easier to delete a specific node given its pointer because the predecessor is immediately known.
- **Disadvantages:** Higher space overhead due to the extra pointer in every node.

- **Applications:** Navigation systems (e.g., "Next" and "Back" in browsers), LRU cache implementations, and Undo/Redo sequences.

3. Circular Linked List (CLL)

- **Structure:** Similar to SLL or DLL, but the **next pointer of the last node points back to the first node**, forming a continuous loop.
- **Operations:** Often managed by an external pointer to the **last node**, which allows $O(1)$ access to both the front (`last->next`) and the rear (`last`).
- **Advantages:** Any node can be a starting point for full list traversal. It eliminates the need for NULL pointers, simplifying certain boundary conditions.
- **Disadvantages:** Infinite loops can occur if the traversal logic is not carefully implemented.
- **Applications:** Round-robin CPU scheduling, buffering data streams, and advanced structures like **Fibonacci Heaps**.

Summary Comparison Table

Feature	Singly Linked List	Doubly Linked List	Circular Linked List
Pointers per Node	1 (Next)	2 (Next, Prev)	1 or 2
Traversal	Forward only	Forward & Backward	Forward (or both if Doubly Circular)
End of List	Points to NULL	Points to NULL	Points back to Head
Tail Insertion	$O(n)$	$O(1)$	$O(1)$ (if last pointer is kept)
Space Complexity	$O(n)$ (Lower overhead)	$O(n)$ (Higher overhead)	$O(n)$
Typical App	Simple Stack/Queue	Browser History	Task Scheduling/Heaps

G.Define a tree and a binary tree. Explain the basic terminology of trees, including root, parent, child, leaf, degree, depth, height, and subtree. Explain the types, advantages, disadvantages, and applications of binary trees

Definition of a Tree and Binary Tree

- **Tree:** A tree is a non-linear, hierarchical data structure consisting of a finite set of one or more data items (nodes). Theoretically, it is defined such that there is a special node called the **root**, and the remaining nodes are partitioned into disjoint subsets, each of which is itself a tree, known as a **subtree**. In graph theory, a tree is defined as a connected undirected graph with no simple circuits or cycles.

Example:



Here, node `A` has 3 children: `B`, `C`, and `D`.

Node `C` has 2 children: `E` and `F`.

There is **no restriction that a node can have only two children**.

- **Binary Tree:** A binary tree is a specific type of tree where each node has a maximum **degree of two**. It is defined as a finite set of data items that is either empty or consists of a root and two disjoint binary trees called the **left subtree** and the **right subtree**.

These children are specifically called:

- **Left child**
- **Right child**

Example:





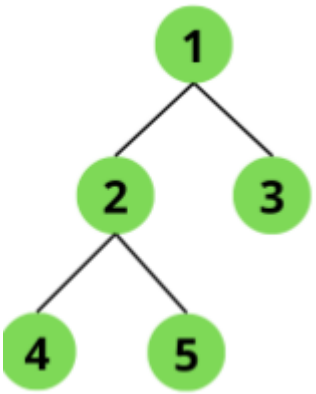
Each node has at most two children.

Basic Terminology of Trees

- **Root:** The first node in the hierarchical arrangement; it is the unique node in the tree that has no parent.
- **Parent:** A node that has one or more successors (children) connected to it by directed edges.
- **Child:** A node that is the immediate successor of another node (its parent).
- **Leaf (Terminal/External Node):** A node that has no children, meaning its degree is zero.
- **Degree of a Node:** The number of subtrees or children attached to a specific node.
- **Degree of a Tree:** The maximum degree among all the nodes within a given tree.
- **Depth:** The depth of a node is the length of the unique path from the root to that node; the root itself has a depth of 0.
- **Height:** The height of a node is the maximum path length from that node to a leaf node. The **height of a tree** is the height of its root node, which is also equal to the maximum level of any node in the tree.
- **Subtree:** A portion of the tree structure consisting of a node and all of its descendants, which themselves satisfy the definition of a tree.

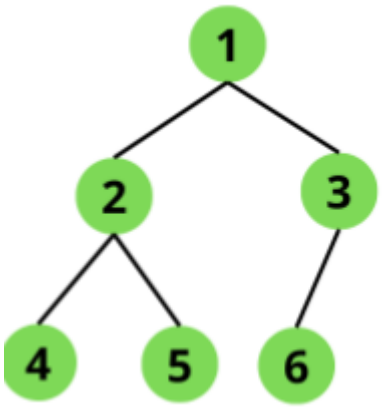
Binary Trees: Types, Advantages, Disadvantages, and Applications
Types of Binary Trees

- **Strictly (Full/Proper) Binary Tree:** A tree where every node has either zero or two children; no node has only one child.



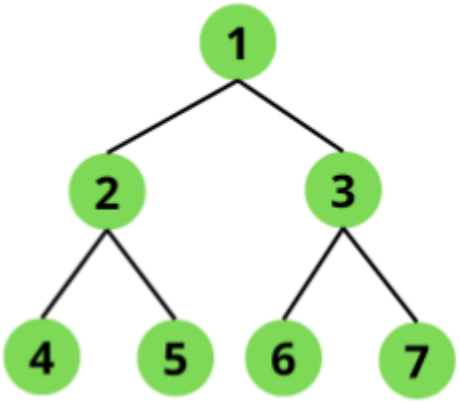
Here, every node has either 0 or 2 children.

- **Complete Binary Tree:** A tree where every level is completely filled except possibly the last level, and all nodes in the last level are as far left as possible.



The last level is filled from the left, so this is a complete binary tree.

- **Perfect Binary Tree:** A strictly binary tree in which all leaf nodes are at the exact same depth.



- **Binary Search Tree (BST):** A binary tree in which for any node ‘n’
    - All values in its left subtree will be lesser than or equal to n
    - All values in its right subtree will be greater than or equal to n

  No values in BST will be repeated

- **AVL Tree:** A self-balancing binary search tree where the heights of the left and right subtrees of any node differ by no more than one.
- **Binary Heap:** A complete binary tree that satisfies the **heap property** (Max-Heap or Min-Heap), used primarily for priority queues.

Advantages

- **Efficiency:** Trees are ideal for representing hierarchical information and allow for faster operations than linear structures.
- **Logarithmic Performance:** In balanced structures like BSTs, operations such as searching, insertion, and deletion take $O(\log n)$ time, whereas they take linear time in linked lists.
- **Dynamic Memory:** Linked representations of binary trees allocate memory only as needed, avoiding the wastage associated with pre-allocated static arrays.

Disadvantages

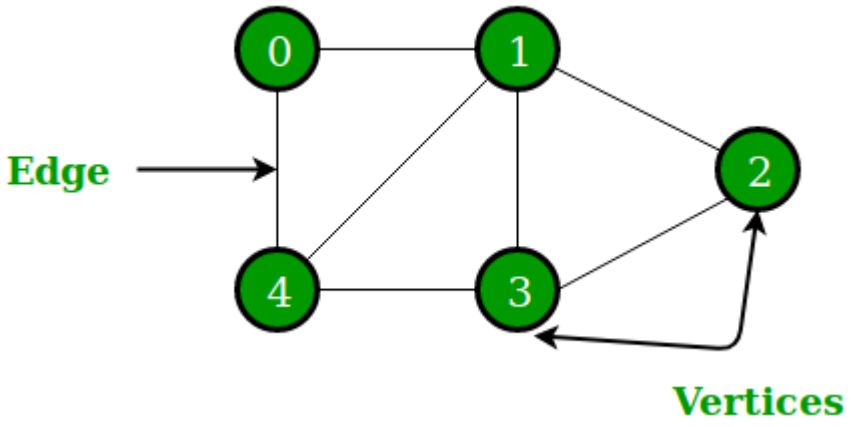
- **Complexity:** The implementation of tree structures and their associated algorithms is more complex than that of linear data structures.
- **Maintenance:** To maintain optimal performance, trees often require complex rebalancing operations (e.g., rotations in AVL or Red-Black trees).
- **Space Overhead:** Linked representations require additional memory in each node to store pointers to children and parents.
- **Array Wastage:** If a binary tree is represented using an array and is not complete, significant memory may be wasted on empty slots.

Applications

- **File Systems:** Representing directory and folder structures on computer systems.
- **Database Indexing:** B-Trees and B+ Trees are used in relational databases like MySQL and PostgreSQL to manage large datasets on disk.
- **Expression Evaluation: Expression trees** are used by compilers to represent and evaluate arithmetic and logical expressions.
- **Decision Making: Decision trees** model algorithms that make a series of binary choices, often used to prove lower bounds for sorting.
- **Priority Management:** Heaps are used to implement priority queues and the efficient Heapsort algorithm.
- **Data Compression:** Huffman coding uses binary trees to create optimal prefix-free codes for data compression

H.Define a graph and explain its fundamental terminology and different types, including directed, undirected, weighted, and unweighted graphs, with suitable examples.

A **graph** $G=(V,E)$ is a discrete, non-linear data structure consisting of a finite, non-empty set of **vertices** (or nodes), V, and a set of **edges**, E, that connect pairs of these vertices. While trees represent hierarchical relationships, graphs are more general structures where nodes can have multiple incoming edges and any pair of elements can be connected.



Fundamental Graph Terminology

- **Vertices and Edges:** Vertices are the discrete points representing elements (like states in a search or operations in a neural network), while edges represent the relationships, models of process, or links between them.
- **Endpoints:** The two vertices associated with a specific edge are called its endpoints.
- **Adjacency and Incidence:** Two vertices are **adjacent** if they are connected by an edge; in this case, the edge is said to be **incident** on those vertices.
- **Degree:** In an undirected graph, the **degree** of a vertex is the total number of edges incident to it. A vertex with a degree of zero is **isolated**, while a vertex with a degree of one is **pendant**.

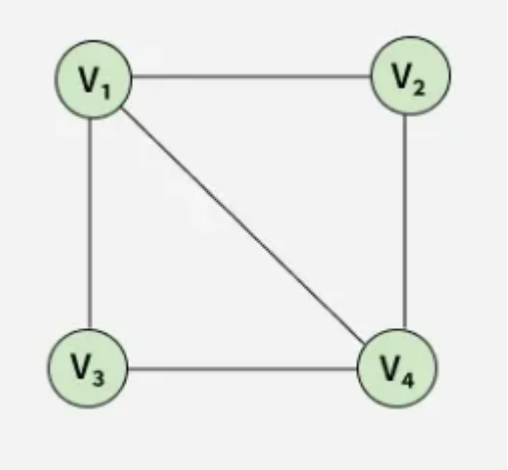
- **Path:** A finite alternating sequence of vertices and edges that begins and ends with vertices is called a **walk**; a **path** is a specific type of walk where neither vertices nor edges are repeated.
- **Cycle:** A **cycle** (or circuit) is a path that starts and ends at the same vertex.
- **Connectivity:** A graph is **connected** if there is a path between every pair of vertices in the structure.

Different Types of Graphs

1. Undirected Graphs

In an **undirected graph**, edges are represented as unordered pairs of vertices, meaning the relationship is symmetric. If an edge exists between vertex u and v, then u is a neighbor of v, and vice versa.

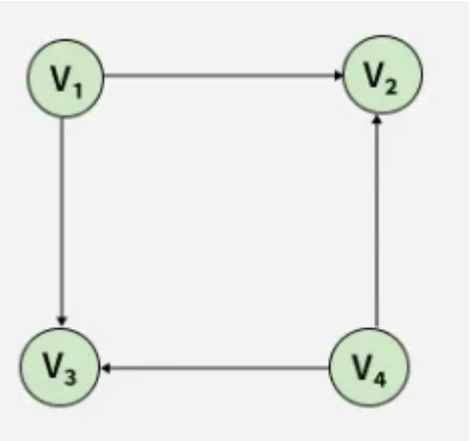
- **Example:** A road network where highways allow travel in both directions can be modeled as an undirected graph.



2. Directed Graphs (Digraphs)

In a **directed graph**, each edge (often called an **arc**) is an ordered pair (i,j), marked by a direction from the initial vertex i to the terminal vertex j.

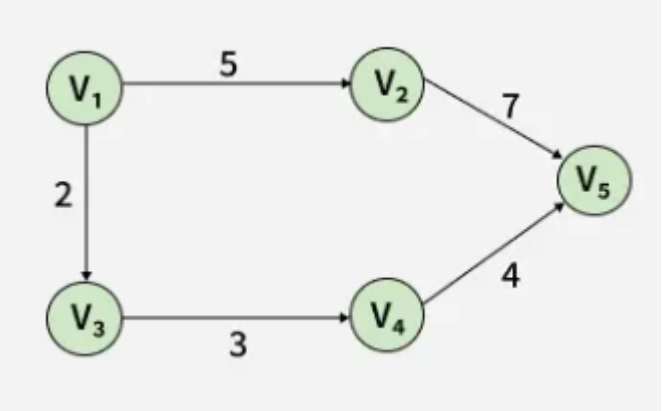
- **Degrees:** These graphs use **indegree** (number of edges entering a vertex) and **outdegree** (number of edges leaving a vertex).
- **Example:** A subproblem graph in dynamic programming (like the rod-cutting problem) where a directed edge from x to y indicates that solving x requires knowing the solution to y.



3. Weighted Graphs

A graph is **weighted** if every edge is assigned a numerical value or "weight". These weights can represent physical measures such as distance, cost, time, or capacity.

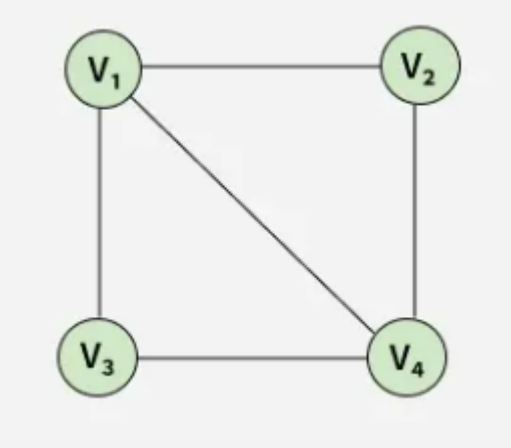
- **Example:** A road map where intersections are vertices and edges are road segments, with edge weights representing the actual road distances between them.



4. Unweighted Graphs

An **unweighted graph** is one where all edges are considered to have a uniform or unit weight. When finding the shortest path in such a graph, the algorithm simply seeks the path with the fewest edges.

- **Example:** A social network like LinkedIn can be viewed as an unweighted graph where nodes are users and edges are connections; finding the "shortest path" identifies the minimum number of intermediaries between two professionals.



I. Differentiate between adjacency matrix and adjacency list. Explain their representations, advantages, disadvantages, space requirements, and suitable applications with examples

Feature	Adjacency Matrix	Adjacency List (ACL/Capabilities)
Representation	2D Array/Matrix	Array of Linked Lists
Space Complexity	$O(V^2)$ (Proportional to $N \times M$)	$O(V+E)$ (Proportional to edges)
Best For	Dense graphs with many connections.	Sparse graphs (most real-world systems).
Edge Lookup	$O(1)$ (Constant time)	$O(degree(v))$ (Linear scan of list)
Primary OS Example	Theoretical Protection Matrix	Access Control Lists (Windows/UNIX)

1. Adjacency Matrix Representation

In a general graph of N nodes, an adjacency matrix is an $N \times N$ structure where each cell $[i,j]$ indicates the presence of an edge between node i and node j.

- **OS Context:** This is best exemplified by the **protection matrix**, where rows represent domains (subjects) and columns represent objects (files, printers,

etc.). Each cell specifies the access rights (edges) that a domain has for a particular object.

- **Space Requirements:** The matrix requires space proportional to the number of nodes squared ($N \times M$ or N
- 2
-).
- **Advantages:**
    - **Fast Lookup:** The system can immediately determine if an interaction (e.g., access) is permitted by checking a single cell.
    - **Conceptual Simplicity:** It provides a clear, unified view of all relationships in the system.
- **Disadvantages:**
    - **Wasted Space:** Modern systems are typically **large and sparse**; most domains have no access to most objects, resulting in a matrix filled primarily with empty cells.
    - **Inflexibility:** Scaling the matrix to include new nodes or objects can be computationally expensive.

2. Adjacency List Representation

An adjacency list stores only the active connections (edges) for each node, typically using a linked list or similar dynamic structure.

- **OS Context:** The sources describe two primary "list" versions of the protection matrix:
    - **Access Control Lists (ACLs):** Storing the matrix by columns, where each object maintains a list of domains that can access it.
    - **Capability Lists:** Storing the matrix by rows, where each domain maintains a list of objects it is authorized to use.
- **Space Requirements:** Space is proportional only to the number of active edges (non-empty elements), making it significantly more efficient for sparse graphs.
- **Advantages:**
    - **Efficiency:** It avoids the "waste of disk space" inherent in storing large, empty matrices.
    - **Dynamic Scaling:** New edges or nodes can be added easily using linked list primitives without reallocating a massive structure.
- **Disadvantages:**
    - **Lookup Overhead:** Determining a specific relationship (e.g., "Does Domain A have access to Object B?") may require scanning the entire list for that node, which is an $O(n)$ operation for n entries in that specific list.

J. Explain how the Divide and Conquer technique is used to solve problems ? Also ,Explain how does the Greedy Algorithm work?

Divide and Conquer Technique

The **divide-and-conquer** method is a powerful algorithm design strategy based on a **recursive structure**. To solve a given problem, the technique involves three characteristic steps at each level of recursion:

1. **Divide:** The original problem is broken down into a number of **subproblems** that are smaller instances of the same problem.
2. **Conquer:** The subproblems are solved **recursively**. However, if the subproblems are small enough (referred to as the **base case**), they are solved directly in a straightforward manner without further recursion.
3. **Combine:** The solutions to the subproblems are merged or combined to create the final solution for the original problem.

A classic example of this approach is **Merge Sort**, where an array is divided into two halves, each half is sorted recursively, and the two sorted halves are then merged back together. Other common examples include **Quick Sort** and **Binary Search**. While some algorithms like Merge Sort require a significant combine step, others like Quick Sort perform most of the work during the divide (partitioning) phase and may not require an explicit combine step.

Working of the Greedy Algorithm

A **greedy algorithm** is a straightforward design technique used for **optimization problems**. It constructs a solution piece by piece through a sequence of steps, making a choice at each stage.

Core Principle

The fundamental principle of a greedy algorithm is to always make the **locally optimal choice**—the choice that looks best at the immediate moment—in the hope that this will lead to a **globally optimal solution**. This "take what you can get now" strategy ignores future consequences or previous choices once they are made.

Operational Steps

1. **Initialization:** The algorithm starts with the initial state of the problem and an empty solution set.
2. **Iterative Selection:** At each step, all possible options available from the current state are evaluated.
3. **Feasibility Check:** The algorithm chooses the best immediate option based on a specific optimization measure (objective function). It then checks if including this choice violates any problem constraints; if it is **feasible**, it is added to the partial solution.
4. **Solution Augmentation:** The chosen item is added to the solution set, and the algorithm moves to a new state.
5. **Termination:** This process repeats until the entire problem is solved or no further choices remain.

Requirements for Optimality

A greedy algorithm is guaranteed to find a globally optimal solution only if the problem possesses two key properties:

- **Greedy-choice Property:** A global optimum can be reached by making a series of locally optimal choices.
- **Optimal Substructure:** An optimal solution to the problem contains within it optimal solutions to its subproblems.

While greedy algorithms are efficient and easy to implement, they do not always yield the optimal solution for every problem, such as in the **0-1 Knapsack problem**, where a dynamic programming approach is often required instead. Popular successful examples include **Dijkstra's algorithm** for shortest paths, **Kruskal's algorithm** for minimum spanning trees, and **Huffman coding** for data compression.

k.Define the Backtracking technique and Explain how the N-Queens problem can be solved using the Backtracking technique. Write the algorithm, illustrate the solution process, and analyze its complexity

Definition of Backtracking

Backtracking is a systematic algorithmic technique used for searching a set of solutions or finding an optimal solution that satisfies specific constraints. It works by building a solution vector one component at a time and using a **criterion (or bounding) function** to test if the partial solution has any chance of success. If the algorithm determines that a partial solution cannot lead to a valid final state, it abandons that path, "backtracks" to the previous decision point, and tries a different alternative. This process of eliminating a large group of unfeasible possibilities in a single step is known as **pruning**. Effectively, backtracking is a refined brute-force approach implemented via recursion that explores a state-space tree to find all (or the best) configurations.

Solving the N-Queens Problem

The **N-Queens problem** is a classical combinatorial challenge where the goal is to place n queens on an $n \times n$ chessboard such that no two queens attack each other. This means no two queens can reside on the same row, column, or diagonal.

To solve this using backtracking:

1. **Representation:** Solutions are represented as an n-tuple $(x$
2. 1
3.
4. $,x$
5. 2
6.
7. $,\ldots,x$
8. n
9.

10.), where x
11. i
12.
13. indicates the column position of the queen placed on row i.
14. **Incremental Construction:** The algorithm places queens row by row, starting from the first row.
15. **Constraint Checking:** For each row, the algorithm tries every column (1 to n) and checks if the position is safe using a bounding function.
16. **Recursive Step:** If a safe position is found, the algorithm moves to the next row; if no safe position exists in the current row, it backtracks to the previous row to move that queen to a different column.

Algorithm (Pseudocode)

The following pseudocode demonstrates the recursive backtracking approach to the N-Queens problem:

Main Algorithm:

```
rnQueens(k, n)
1. For col[k] = 1 to n
2.    If positionOK(k) then
3.       If k == n then
4.          Print the solution (col[1...n])
5.       Else
6.          rnQueens(k + 1, n)
```

Bounding Function (positionOK):

```
positionOK(k)
1. For i = 1 to k - 1
2.    If (col[k] == col[i]) OR (abs(col[k] - col[i]) == k - i)
3.       Return false // Same column or same diagonal
4. Return true
```

Illustration of the Solution Process (4-Queens)

Using a 4×4 board as an example, the process follows these steps:

- **Step 1:** Place Queen 1 at (Row 1, Col 1).
- **Step 2:** Attempt to place Queen 2 in Row 2. Columns 1 and 2 are attacked. Queen 2 is placed at (Row 2, Col 3).
- **Step 3:** Attempt Row 3. Every column (1-4) is attacked by either Queen 1 or Queen 2.
- **Step 4 (Backtrack):** Move Queen 2 to the next available safe spot, (Row 2, Col 4).
- **Step 5:** Attempt Row 3 again. Queen 3 can now be placed at (Row 3, Col 2).
- **Step 6:** Attempt Row 4. Every column is attacked.
- **Step 7 (Backtrack):** Since Row 3 has no more options, backtrack further to Row 1 and move Queen 1 to (Row 1, Col 2).

- **Step 8:** The process continues until a valid 4-tuple solution, such as $(2,4,1,3)$, is reached.

Nodes that fail the bounding function are marked as **"dead nodes"** and "killed," preventing the generation of their children in the state-space tree.

Complexity Analysis

- **Time Complexity:** The total number of possible n-tuples in a brute-force search is n
- n
- (e.g., 8
- 8
- for 8-Queens). However, because each queen must be in a unique column, the backtracking search space is reduced to $O(n!)$ permutations. While pruning significantly reduces the actual number of nodes visited, the worst-case complexity remains exponential.
- **Space Complexity:** The algorithm primarily uses space for the recursion stack and the array storing queen positions, resulting in a space complexity of $O(n)$. This is much more efficient than breadth-first approaches that might require storing large portions of the state-space tree